Phase 2 — Grade 4: Supervised Learning

Detailed Revision Guide — full formula breakdowns, edge cases, and common-mistake debugging

1. Supervised Learning — The Setup

Supervised learning means learning from examples where the correct answer is already known, so the model learns the pattern linking input (features) to output (label/target).

Core vocabulary

```
Features = the measurable inputs, e.g. [size, bedrooms, location_score] for a house

Label/Target = the known correct answer for each example, e.g. the house's actual sale price

Regression = predicting a NUMBER. Classification = predicting a CATEGORY.
```

EDGE CASE: A feature vector is literally a vector from Linear Algebra (Grade 1)
[size, bedrooms, location_score] is exactly the house-vector from Grade 1's Vectors sub-topic. A full dataset of many houses is exactly a MATRIX (vectors stacked as rows). Supervised learning is finding a pattern connecting each matrix row to its known label.

1.1 Train/Test Split

If you test a model on the SAME data it trained on, you cannot tell whether it learned the real underlying pattern, or simply MEMORIZED those specific examples (like a student memorizing an answer key instead of understanding the material).

The fix

```
Training set (typically 70-80% of data): what the model actually learns from

Test set (the rest): held back, NEVER shown during training, used only afterward to check real generalization to genuinely new data
```

COMMON MISTAKE: Assuming high training-score = good model
99% accuracy on TRAINING data could mean (1) the model genuinely learned the pattern, OR (2) it just memorized those exact examples, including noise and coincidences. Both cases produce the SAME high training score -- you cannot tell them apart without testing on data the model never saw. This failure mode (memorizing instead of generalizing) is called OVERFITTING, covered in more depth in a later grade.

EDGE CASE: Data leakage: when the test set isn't really 'unseen' (a real-world gotcha, not yet covered in depth)
If any test-set information accidentally influences training (e.g. duplicate rows appearing in both sets, or statistics computed using the full dataset before splitting), the test score becomes artificially inflated and no longer proves real generalization. Always split BEFORE any data processing that uses statistics from the data (like scaling), not after.

2. Linear Regression

Formula

```
y-hat = m×x + b
```

```
x = input feature. y-hat ('y-hat') = the model's PREDICTED output (the hat marks 'prediction', not the real value). m = slope (rate of change). b = intercept (baseline value when x=0).
```

TIP: Direct callback to Calculus (Grade 2)
y=mx+b is the EXACT example used when teaching the constant rule -- m is a rate of change (a derivative-flavored idea), b is the constant/intercept. Training linear regression means finding the best m and b.

2.1 The Real Problem Being Solved (a common point of confusion)

There are two DIFFERENT situations that are easy to mix up:

Situation 1 vs. Situation 2

```
SITUATION 1 (just USING a formula): if m and b are ALREADY known, you simply plug in x to get y-hat. Nothing to 'figure out'.
```

```
SITUATION 2 (the ACTUAL regression problem): you do NOT know m and b. All you have is real, messy data (x, actual-y pairs), and you must DISCOVER what m,b best explain that data.
```

COMMON MISTAKE: Confusing Situation 1 and Situation 2
It's natural to assume m,b are given, since the formula ships with letters already in place. But regression ALWAYS starts from Situation 2 in real use -- raw observed data, with unknown m,b you must search for. Before treating any m,b as fixed, check: were these values given directly, or are they what the problem is asking you to find?

2.2 Why There's No Single 'Perfect' m With Real (Noisy) Data

Worked proof: hours studied vs actual score

```
Data: (0,22), (1,29), (2,41), (3,48)
```

```
Slope between (0,22) and (1,29): m = (29-22)/(1-0) = 7
```

```
Slope between (1,29) and (2,41): m = (41-29)/(2-1) = 12
```

```
Slope between (2,41) and (3,48): m = (48-41)/(3-2) = 7
```

```
Three DIFFERENT answers (7, 12, 7) depending on which pair is used -- no single m fits every pair exactly, because real data has noise/randomness.
```

EDGE CASE: b CAN be read directly off the data, but only in the special case x=0 is actually present
In this example, b=22 matched exactly only because the data happened to include a row at hours=0. If no row had x=0, b would need to be estimated the same way m is -- found via fitting, not read off directly.

3. MSE — Measuring How Wrong a Guess Is

Formula

Error (per point) = Actual - y-hat

SSE (Sum of Squared Errors) = $\sum$ (Actual - y-hat)^2

MSE (Mean Squared Error) = SSE / n (n = number of data points)

Worked Example: guess m=9, b=22 on the 4-point dataset

hours=0: y-hat=22, actual=22, error=0

hours=1: y-hat=31, actual=29, error=-2

hours=2: y-hat=40, actual=41, error=1

hours=3: y-hat=49, actual=48, error=-1

SSE = $0^2+(-2)^2+1^2+(-1)^2$ = 0+4+1+1 = 6

MSE = 6/4 = 1.5

COMMON MISTAKE: Assuming summing raw errors is good enough

Two models with errors [+10,-10] and [+1,-1] both sum to exactly 0 under plain addition -- making a WILDLY wrong model look identical to a barely-wrong one. This is the same cancellation problem solved in Grade 3's VARIANCE formula (squaring deviations from the mean). MSE applies the identical fix: square every error first, so +10 and -10 both become 100, correctly showing both as equally bad, instead of appearing to cancel to zero.

EDGE CASE: MSE = 0 only when every single prediction is exactly correct

SSE=0 requires every individual (Actual-y-hat) term to be exactly 0, since each term is squared (always ≥0) before summing -- a single non-zero error, however small, makes the whole SSE strictly positive. Perfect MSE=0 fits are rare outside of toy/clean datasets like the very first example in this grade.

TIP: MSE is a FUNCTION of (m,b), not one fixed number

Changing the guess from (m=9,b=22) to (m=10,b=22) on the SAME data changes every prediction, every error, and therefore MSE itself (1.5 vs 6.5 in the worked examples) -- MSE must be recalculated fresh for every different (m,b) combination tried, which is exactly why training needs a systematic search method (gradient descent), not a single calculation.

4. Gradient Descent

Picture MSE as ALTITUDE on a hill, where m and b are your position (exactly like x,y on the hillside from Grade 2). High MSE = high up the hill (bad). Training means taking steps DOWNHILL until reaching the lowest point.

4.1 Why Moving Opposite the Gradient Goes Downhill (numeric proof)

Reusing the exact Grade 2 hillside numbers: f(x,y)=x^2+y^2, gradient at (3,4) = [6,8]

Moving in the POSITIVE direction (x:3->3.01, y:4->4.01, roughly): altitude INCREASES (uphill)

Moving in the NEGATIVE direction (x:3->2.99, y:4->3.99):

f(2.99,3.99) = $2.99^2+3.99^2$ = 8.9401+15.9201 = 24.8602

Compare to f(3,4)=25: 24.8602 < 25 -> altitude DECREASED (downhill)

Conclusion: the gradient's sign points uphill; its NEGATIVE points downhill.

4.2 The Update Rule

Formula

m_new = m_old - (learning_rate) × ∂MSE/∂m

b_new = b_old - (learning_rate) × ∂MSE/∂b

The MINUS sign moves opposite to the gradient (downhill, toward lower MSE).

Learning rate = a small positive number controlling step size.

EDGE CASE: Learning rate too large or too small (flagged, not yet covered in depth)
Too LARGE a learning rate can overshoot the minimum, bouncing around or even increasing MSE over time (divergence) instead of converging. Too SMALL a learning rate converges correctly but very slowly, needing far more steps. Choosing a good learning rate is itself a real tuning problem addressed more fully in later grades.

EDGE CASE: When does gradient descent STOP? (the natural stopping condition)
When ∂MSE/∂m AND ∂MSE/∂b both equal (or are very close to) 0, you're at a flat point on the MSE-surface -- no direction is uphill or downhill anymore, meaning you've reached (at least) a minimum. In practice, training stops after a fixed number of steps or once the gradient's size drops below a small threshold, rather than waiting for it to hit exactly 0.

4.3 Deriving ∂MSE/∂m (the chain rule, applied for real)

The nested structure

MSE = (1/n) ∑ (Actual - y-hat)^2, where y-hat = mx+b

Let u = Actual - (mx+b) (the error itself -- this is the INNER function)

Outer function: square u. Chain rule: d(u^2)/du = 2u

Inner derivative w.r.t. m (treating Actual, x, b as constants): ∂u/∂m = -x

Multiply (chain rule): ∂(u^2)/∂m = 2u × (-x) = -2x(Actual-y-hat)

Averaged over n points: ∂MSE/∂m = (-2/n) ∑ x×(Actual-y-hat)

By the same process: ∂MSE/∂b = (-2/n) ∑ (Actual-y-hat) (no x multiplied in)

Worked Example: verifying the formula against guess m=9,b=22

x=0,error=0: x×error=0. x=1,error=-2: 1×-2=-2. x=2,error=1: 2×1=2.
x=3,error=-1: 3×-1=-3

∑x×error = 0-2+2-3 = -3

∂MSE/∂m = (-2/4)×(-3) = 6/4 = 1.5

∑error = 0-2+1-1 = -2

∂MSE/∂b = (-2/4)×(-2) = 4/4 = 1.0

COMMON MISTAKE: Sign errors when multiplying two negatives (a genuine slip from practice)
Computing ∂MSE/∂b = (-2/4)×(-2), the answer was first given as -1 instead of the correct +1. Negative × negative = positive -- a small but consequential slip, since the SIGN of the gradient determines which direction (increase or decrease) the next update step moves in. Always double-check the sign of a product separately from its magnitude, especially with two negative numbers.

TIP: Numeric nudge check confirms the formula, without needing the full symbolic derivation
Nudging m from 9 to 9.01 directly (recomputing all 4 predictions, errors, and MSE by hand) gave a rate of roughly 1.535 -- closely matching the formula's exact 1.5. If a symbolic chain-rule derivation feels abstract, this direct nudge-and-compare approach (same technique used throughout Grade 2) always works as a sanity check or alternative route to the same answer.

Worked Example: one full gradient descent step, learning_rate=0.1

```
m_new = 9 - 0.1×1.5 = 9 - 0.15 = 8.85

b_new = 22 - 0.1×1.0 = 22 - 0.1 = 21.9

Both gradients were POSITIVE (uphill), so both m and b correctly moved
DOWNWARD.
```

5. Full Debugging Checklist for This Grade

---

1. Am I in 'Situation 1' (using known m,b) or 'Situation 2' (discovering unknown m,b from data)? 2. Have I checked that my test set was truly held out and never influenced training? 3. Before computing MSE, have I subtracted to get the error (Actual-y-hat) for EVERY point, not just assumed a value? 4. Have I squared EVERY error before summing, never summing raw (signed) errors directly? 5. Am I recalculating MSE fresh for each new (m,b) guess, not reusing an old MSE value? 6. In the gradient formula, have I included the MINUS sign in the update rule (subtracting, not adding, the gradient)? 7. When multiplying two negative numbers in a gradient calculation, have I double-checked the sign separately from the magnitude?